```
    results = []
    for s in out.split('\n\n'):
        #print(s)
        a = re.findall('Address: .*Name: ([A-Za-z\s\.]+).*School: ([A-Za-z\s,]+).*Street: (\d+)\s+([A
        if a:
            a = a[0]
            #print(a)
            results.append(MailingAddress(a[0].strip(), a[1].strip(), int(a[2].strip()), a[3].strip()
    return results
    #raise NotImplementedError()

#print(generate_addresses())
```

0.0s Python

```
# Invoke student code
from contextlib import redirect_stderr
import tempfile

with redirect_stderr( tempfile.TemporaryFile('wt') ) as error_catcher:
    results=generate_addresses()

# Verify length
assert (
    len(results) == 5
), f"You did not return five and only five results, instead we got {len(results)}."
```

2m 48.2s Python

```
#results
#len(results)
```

Python

Python

Grades

Close

Passed·Grade Received: 100%

Submissions

July 22, 2024 10:32 PM IST (Highest Grade)

Assignment (Score: 3.0 / 3.0)

1. Test cell (Score: 1.0 / 1.0)
2. Test cell (Score: 1.0 / 1.0)
3. Test cell (Score: 1.0 / 1.0)

Assignment¶

I often need "fake data" to show people how to do data manipulation tasks with regular expressions or pandas. The problem is that sometimes the data I generate on the web is too messy, and I get bogged down showing students how to clean all of the data when some of it isn't representative of what I want. In this assignment you get a chance to help me generate realistic fake data, all with llama 2!

For each question I will describe the data in natural language and you must write a function which queries llama 2 to generate data in that format and adhere to the description I've written.

Question 1¶

Generate for me a list of ten fictitious names, where the firstname is a single word, and the last (family) name may be (but doesn't have to be!) up to two words separated by a hyphen. Don't include titles, honorifics, or middle names. The autograder will expect that you return a list[str] where each value in the list is a full name.

In [1]:

```
import os
import re
from llama_cpp import Llama
from llama_cpp.llama_types import *
from llama_cpp.llama_grammar import *
```

In [2]:

```
#print(sorted(list(os.environ.keys())))
```

In [3]:

Student's answer (Top)

```
def generate_names() -> list[str]:
    results: list[str] = []

    # YOUR CODE HERE
    # Import the llama.cpp python bindings and load the model
    model: Llama = Llama(model_path=os.environ["LLAMA_13B"], verbose=False, n_ctx=4096)
    prompt = '''
    Sandipan Dey
    Elton John
    Rivest Samir-Addleman
    '''
    grammar= r'''
    root ::= (name "\n" name "\n" name "\n" name "\n" name "\n" name "\n" name "\n" name "\n" name "\n")
    sentence ::= [A-Z][a-z]*
    # A name should be a sentence space sentence - sentence.
    name ::= sentence " " sentence "-" sentence
    '''
    response = model.create_completion(prompt, grammar=LlamaGrammar.from_string(grammar=grammar), max_tokens=4096, stream=True) #top_k=10,
    out = ''
    for item in response:
        #print(item['choices'][0]['text'], end="")
        out += item['choices'][0]['text']
    results = out.split('\n')[:10]
    return results
    #raise NotImplementedError()

#print(generate_names())
```

In [4]:

Grade cell: `q1_tests` Score: 1.0 / 1.0 (Top)

```
# Invoke student code
from contextlib import redirect_stderr
import tempfile

with redirect_stderr( tempfile.TemporaryFile('wt') ) as error_catcher:
    results = generate_names()

# Verify the length
assert len(results)==10, f"You did not return ten and only ten results, instead we got {len(results)}."
```

In [5]:

```
#results
```

Question 2¶

Generate for me a list of 5 things to do in your hometown (or mine if you prefer, Ann Arbor Michigan!). The key is that these should all (a) start with a number and (b) be no more than three sentences long. So the following would be a good item:

- 1. Go to the Henry Ford Museum. The Henry Ford Museum has all sorts of wonderful exhibits for all ages. One particular highlight includes giants trains!

While the following would not be good items (the first item does not start a numbered list, the second item is not a sentence as it doesn't end in punctuation, and the third item just goes on and on and on):

- A. Go to the University of Michigan. The University of Michigan is a school with more than 50,000 students in Ann Arbor, MI. The University of Michigan is a public School.
- 2. Visit the Detroit Eastern Market
- 3. Visit Sleeping Bear Dunes. The dunes are located along the northwest coast of the Lower Peninsula of Michigan in Leelanau and Benzie counties near Traverse City. It covers a 35-mile-long stretch of Lake Michigan's eastern coastline, as well as North and South Manitou islands. This national park is known for its massive dunes, some of which are over 400 feet high. The area gets its name from the Native American legend of the Sleeping Bear. According to the story, a mother bear and her two cubs were trying to cross Lake Michigan from Wisconsin to escape a forest fire.

In [6]:

Student's answer (Top)

```
def generate_trip_recommendations() -> list[str]:
    results: list[str] = []

    # YOUR CODE HERE
    model: Llama = Llama(model_path=os.environ["LLAMA_13B"], verbose=False, n_ctx=4096)
    prompt = """
    1. Go to the Henry Ford Museum. The Henry Ford Museum has all sorts of wonderful exhibits for all ages. One particular highlight includes giants tra
    2. Go to the University of Michigan. The University of Michigan is a school with more than 50000 students in Ann Arbor MI. The University of Michiga
    """

    grammar = r"""
    root ::= ("1. " item "\n" "2. " item "\n" "3. " item "\n" "4. " item "\n" "5. " item)
    item ::= sentence [ ] sentence [ ] sentence
    sentence ::= [A-Z] [A-Za-z0-9 ,-]* ("." | "!" | "?")
    """

    response = model.create_completion(prompt, grammar=LlamaGrammar.from_string(grammar=grammar), max_tokens=4096, stream=True) #, temperature=0) #top_k
    out = ''
    for item in response:
        out += item['choices'][0]['text']
    results = out.split('\n')[:5]
    return results

    #raise NotImplementedError()

#print(generate_trip_recommendations())
```

In [7]:

Grade cell: `q2_tests` Score: 1.0 / 1.0 (Top)

```
# Invoke student code
from contextlib import redirect_stderr
import re
import tempfile

with redirect_stderr( tempfile.TemporaryFile('wt') ) as error_catcher:
    results=generate_trip_recommendations()

# Verify length
assert len(results)==5, f"You did not return five and only five results, instead we got {len(results)}."
```

In [8]:

```
#results
```

Question 3¶

Generate for me US-based addresses which have a person's name which usually appears on the first line, an optional company name which often goes on the second line, a street address which has a number followed by some text description, a city and state where the state is a two letter identifier and comes after the city name, and zip code which is a five digit number (but as a string, since it could start with 0) followed by an optional hyphen and four more digits.

To make it easy for you to conform to this set of requirements, I created a simple class from the following example -- my mailing address!

Dr. Christopher Brooks School of Information, University of Michigan 105 S. State St. Ann Arbor, MI 48109-1285

Your function should return exactly 5 of these entries!

And, if you've gotten this far in the course, why not send me a postcard and introduce yourself? Everyone loves getting mail!

In [9]:

Student's answer (Top)

```
from dataclasses import dataclass


@dataclass
class MailingAddress:
    name: str  # Full name, e.g. Dr. Christopher Brooks
    business_name: (
        str #| None
    )  # Optional business name, e.g. School of Information, University of Michigan
    street_number: int  # Numeric address value, e.g. 105
    street_text: str  # Street information other than numeric address, e.g. S. State St.
    city: str  # City name, e.g. Ann Arbor
    state: str  # State name, only two letters, e.g. MI for Michigan
    zip_code_short: str  # The first five digits of the zip code, e.g. 48109, as a string value, since it could start with 0
    zip_code_long: (
        str #| None
    )  # The extended zip code (optional) which is the full zip code, e.g. 48109-1285


def generate_addresses() -> list[MailingAddress]:
    results: list[MailingAddress] = []

    # YOUR CODE HERE
    model: Llama = Llama(model_path=os.environ["LLAMA_13B"], verbose=False, n_ctx=2048)

    # Our prompt will just be a list of Mushroom questions in Aiken format
    prompt='''
        1. Address:
        Name: Dr. Christopher Brooks
        School: School of Information, University of Michigan
        Street: 105 San State Street
        City: Ann Arbor, MI
        Pin: 48109-1285

        2. Address:
        Name: Mr. Sandipan Dey
        School: School of Computer Science, University of Baltimore County
        Street: 20 Hellthorpe Street
        City: Baltimore, MD
        Pin: 12345-1122
    '''

    grammar= r'''
    root ::= ("1. " address "\n" "2. " address "\n" "3. " address "\n" "4. " address "\n" "5. " address "\n")
    # A sentence is just alphanumerica latin values, plus punctuation and whitespace
    # No parentheticals are allowed in a sentence, but a comma and hyphen are
    address ::= ("Address: \n" "Name: " name "\n" "School: " office "\n" "Street: " street "\n" "City: " city "\n" "Pin: " pin "\n")
    # A question should be a sentence or two, no more.
    name ::= [A-Z][A-Za-z .]+
    # An answer is a capital letter followed by a close parens and then a sentence
    office ::= [A-Z][A-Za-z ,]+
    # The question closes with an epilog telling us the question has finished
    street ::= [0-9]+ [ ] [A-Z][A-Za-z .]+
    city ::= [A-Z][A-Za-z .]+ "," [ ] [A-Z][A-Z]
    pin ::=  [0-9]+ "-" [0-9]+
    '''

    # Finally, let's invoke a completion
    result = model.create_completion(prompt,
        grammar=LlamaGrammar.from_string(grammar=grammar),
        stream=True,
        max_tokens=4096)
    out = ''
    for item in result:
        out += item['choices'][0]['text']
    #print(out)
    results = []
    for s in out.split('\n\n'):
        #print(s)
        a = re.findall('Address: .*Name: ([A-Za-z\s\.]+).*School: ([A-Za-z\s,]+).*Street: (\d+)\s+([A-Za-z\s,]+).*City: ([A-Za-z\s]+),\s+([A-Z][A-Z]).*P
        if a:
            a = a[0]
            #print(a)
            results.append(MailingAddress(a[0].strip(), a[1].strip(), int(a[2].strip()), a[3].strip(), a[4].strip(), a[5].strip(), a[6].strip(), a[6].st
    return results
    #raise NotImplementedError()

#print(generate_addresses())
```

In [10]:

Grade cell: `q3_tests` Score: 1.0 / 1.0 (Top)

```
# Invoke student code
from contextlib import redirect_stderr
import tempfile

with redirect_stderr( tempfile.TemporaryFile('wt') ) as error_catcher:
    results=generate_addresses()

# Verify Length
assert len(results)==5, f"You did not return five and only five results, instead we got {len(results)}."
```

In [11]:

```
#results
#len(results)
```

In []:

This assignment was graded by mooc_adswpy:e5e20d3b91dd, v1.45.052423